VelaPaw — On-Device AI Multi-Pet Smart Feeder

Technical Report

2026 Inaugural openVela AI Hardware Developer Contest

Track: AI Hardware Product Innovation

Team: CircuitForge (`contest2026_043`)

Project: VelaPaw

Repository: `contest2026_043_CircuitForge` (branch `dev-ai-contest-2026`)

Hardware: openVela (NuttX) / Waveshare ESP32-S3-Touch-LCD-3.5-C

Date: 2026-09-06

> This is the English edition of the submission technical report. A Chinese edition with identical content is provided as `VelaPaw_技术报告.pdf` / `.docx`. Where the two differ in wording, the Chinese edition is the submitted version of record.

---

1. Deliverables

Item	Required	File	Notes
Work introduction document (this technical report)	Required	`VelaPaw_技术报告.pdf` / `.docx` `VelaPaw_Technical_Report.pdf` / `.docx`	This document. It **is** the work introduction document required by the submission rules — the organiser's official template, filled in, with no format of our own invented. Supplied as PDF and DOCX, in Chinese and English.
Demo video	Required	`VelaPaw_Demo.mp4`	**4 min 58 s** (≤5 min); 1920×1080 / 30 fps, H.264 High + AAC 48 kHz stereo, 93.3 MB, with **burnt-in EN/中文 subtitles**. Shot entirely on real hardware: touchscreen interaction, recognition-gated dispensing, stepper feeding, voice scheduling, and a live console capture of `ai_agent` answering `ask` and pushing proactively (the closing

			console segment is deliberately left unsubtitled so the subtitles do not cover the log text being shown). The video is also embedded in the repository README.
Product photos	Optional	`hardware/preview/`	Renders and photos of the 3D-printed dispenser and full assembly.
Supplementary product introduction	In the repository (not in the archive)	`VelaPaw_Project_Intro.pdf` `VelaPaw_Project_Intro_zh.pdf`	An earlier non-technical product introduction (bilingual) with market and value analysis, kept in the repository for reference. It is **not** part of the submission archive — the template document above is the work introduction document.
Poster / defence deck	Optional	—	Not submitted.
AI-coding logs	Required	`logs/KingMbewe/`	Collected automatically by the organiser-provided `contest-log-collector`; **never hand-edited**. 17 JSONL files, 78.7 MB, 55,255 events.

2. Project Facts

Item	Detail
Work name	VelaPaw — on-device AI multi-pet smart feeder
Team name	CircuitForge (ID `contest2026_043`)
Team members and division of labour	**Kingsley Mbewe (金斯利)**, GitHub `KingMbewe` — **a solo entry; all work on this project is his own.** By area: product definition and mechanical design; the self-designed 3D-printed paddle-rotor dispenser and full assembly; the openVela / NuttX board port and driver work (ST7796 display, OV5640 camera, ES8311 audio, 28BYJ-48 stepper, PCF85063 RTC, USB CDC-NCM networking); the INT8 operator-library port for Xtensa LX7; identity-model training and on-device TFLite-Micro integration; the LVGL bilingual UI and Trends dashboard; the keyword-spotting

	voice-scheduling dialog; `ai_agent` integration and the custom Skill; and all documentation, the demo video and the submission package.
Topic direction (track)	AI Hardware Product Innovation
Hardware	Waveshare ESP32-S3-Touch-LCD-3.5-C (ESP32-S3, dual-core Xtensa LX7, 8 MB PSRAM, 16 MB flash, 320×480 ST7796 capacitive touch panel, OV5640 camera, PCF85063 RTC, ES8311 audio codec) plus a self-designed 3D-printed paddle-rotor dispenser (28BYJ-48 stepper / ULN2003)
OS	openVela (NuttX), board config `esp32s3-devkit:waveshare_lcd`
openVela capability areas used	**Graphics** (LVGL), **AI** (`apps/mlearning/tflite-micro` + the `ai_agent` framework), **Multimedia** (NuttX Audio / I²S / ES8311 + the `apps/video` camera stack) — **all three**
Wake word	**The product has no always-on voice wake feature.** Voice scheduling is started by tapping a button on the "edit pet" page and then answering prompts; it is a button-triggered constrained-vocabulary keyword spotter, not an always-listening wake word. The contest rule mandating *你好，openvela* / *Hello, openvela* therefore does not apply. Should always-on wake be added later, that exact wake word will be used.
Licence	Apache License 2.0
Code hosting	GitHub — `github.com/open-vela/contest2026_043_CircuitForge`, branch `dev-ai-contest-2026`

Abstract

An ordinary timed feeder cannot tell which animal is standing at the bowl, so in a multi-pet household per-pet portioning and per-pet weight control are impossible — and per-pet portioning is the first thing a vet prescribes. VelaPaw, built on openVela (NuttX) and a Waveshare ESP32-S3-Touch-LCD-3.5-C, identifies the individual pet on-device from a single camera and gates dispensing on two conditions together: the pet is recognised, *and* one of its meals is due today and unfed. Recognition uses open-set metric learning rather than classification — the model emits only a 128-dimensional L2-normalised embedding, so enrolling a new pet is a pure runtime operation requiring no retraining and no firmware update. Measured results: porting an INT8 operator library to Xtensa LX7, for which openVela ships none, cut inference from **6.75 s to 1.31 s**; tracing a display fault with a logic analyser to panel bit-desynchronisation allowed a move to 40 MHz hardware SPI with GDMA, cutting full-screen repaint from **246 ms to 61 ms**; and trimming 20

unused tools compressed the `ai_agent` request body from 19,391 B to fit NuttX's IOB pool, taking end-to-end latency from **150–300 s to 3.6–35.6 s**. All image processing and every feeding decision happen on-device and **no image ever leaves the hardware**; the only networked component, `ai_agent`, uses a custom Skill to read the records the feeder writes to littlefs and speaks only when an appetite threshold is crossed. Unplugged from the network, the feeder behaves identically.

3. Technical Report

3.1 Introduction

3.1.1 Background and problem statement

In a multi-pet household, an ordinary timed feeder has a structural flaw: **it does not know which animal is standing at the bowl**. The lid opens on schedule, whoever arrives first eats everything, and the slower one goes hungry. Per-pet portions, per-pet weight control, per-pet appetite monitoring — none of it is possible. And per-pet portioning is precisely the first thing a vet asks for when prescribing a weight-loss, disease-management, or post-operative plan.

Existing camera-equipped feeders — for instance the Xiaomi Mijia Smart Pet Feeder 2 Vision Edition, launched March 2026 at ¥449 crowdfunded / ¥549 retail — solve the problem of letting the owner *see* the bowl; what the camera produces is a video stream. They do not solve *identifying* which animal it is. Such products also bind the camera feed to a cloud account, which is the single largest source of hesitation pet owners have about a household camera.

VelaPaw therefore defines its problem as:

> On an MCU-class device, with **no network, no cloud, and no phone app**, use a single camera to tell which pet is at the bowl, and dispense only the meal *that* pet is due *right now*; then keep a long-term record of each pet's intake and proactively tell the owner when appetite goes wrong.

That last "proactively tell the owner" layer is carried by openVela's `ai_agent` framework and is the **only** part of the system that needs a network. The feeder itself behaves identically offline.

3.1.2 Technical challenges

Challenge	What it looked like
① **Compute for fine-grained on-device recognition**	Individual pet ID is a fine-grained recognition problem requiring a MobileNet-class backbone plus a metric-learning head. Running that model on Xtensa LX7 through TFLite-Micro's *reference* kernels takes **≈6.75 s per inference** — unusable. An early TFLM spike (`docs/TFLM_SPIKE_REPORT.md`, run in QEMU, not on S3) profiled a comparable INT8 CNN and found **CONV_2D accounts for 84% of the clock** (643,819 of 768,105 ticks): latency is almost entirely decided by the INT8 convolution kernels. Yet the openVela tree ships only `cmsis-nn` (ARM) and `nnlib-hifi4` (HiFi DSP) — **there is no operator library for LX7**.

② Adding a pet must not require retraining	No consumer product can ask a user to retrain a model to add a cat. N-way classification had to be abandoned in favour of open-set metric learning, with a threshold that holds up under real household lighting.
③ The model does not fit the MCU's memory map	A 1.44 MB identity model cannot be memory-mapped as a `const` array — the ESP32-S3 DROM0 MMU window saturates well before that. It must be read from raw flash into PSRAM; but **a flash read suspends the CPU cache, and PSRAM is reached *through* that cache**, so if the loading thread's stack lives in PSRAM the whole SoC deadlocks.
④ The hardware-SPI display "renders nothing"	Full-screen writes worked; any **partial window** (a `CASET/RASET` sub-rectangle followed by `RAMWR`) never appeared — while the identical byte sequence over bit-banged GPIO worked perfectly. This survived **roughly 60 rounds of blind rebuilds**.
⑤ ai_agent's request body vs. the network buffer	`ai_agent` registers 36 tools by default, whose JSON schemas push a single LLM request body to **19,391 B**, of which **9,946 B is tool schema**. NuttX's IOB pool is only `NBUFFERS(64) × BUFSIZE(196) = 12,544 B`, leaving roughly **7,840 B** usable after `THROTTLE(24)`. The request is far larger than the available buffer; the symptom is an intermittent hang with no timeout.
⑥ Agent answers that *look* right but are fabricated	The heartbeat replayed its own history; `list_dir`'s prefix match plus a local-tool shortcut handed `(no files found)` straight to the user as a final answer; `ask` was truncated at the seventh word by `MAX_ARGS`. None of these raise an error — they simply make the answer fluent and wrong.

3.1.3 Innovations

1. **Dual-condition "recognition × schedule" gating.** Food is dispensed only when "this is that pet" *and* "one of its meals is due today and has not been fed" are both true — at most once per meal per day, with any meal individually skippable. Recognition alone would feed the greedy one all day; a timer alone would feed whoever happened to walk past. This AND gate is the core product logic and the precondition for per-pet portioning to mean anything.
2. **Open-set metric learning instead of a classifier.** The model is only a generic pet feature extractor, mapping a 128×128 frame to an L2-normalised 128-dimensional embedding. Enrolment = store the mean embedding of a few photos; recognition = cosine similarity. **Adding a pet is a pure runtime operation — no retraining, no firmware update.**
3. **A bug that survived 60 rebuilds turned into a 4× speedup.** A logic analyser disproved the "software problem" hypothesis and reframed it as a panel bit-synchronisation problem (§3.4.3). With that fixed, the display moved from bit-banging to 40 MHz hardware SPI + GDMA: full-screen repaint **246 ms → 61 ms**.
4. **ai_agent as a silent health monitor, not a chatbot.** A heartbeat task periodically reads the records the feeder itself wrote to flash and **speaks only when a threshold is crossed**. When everything is fine, the device says nothing. The custom Skill states explicitly that only numbers from those files may be used, and nothing may be invented.

5. **A clear "offline-first, network-optional" boundary.** Every feeding decision and all image processing happen on-device; **no image ever leaves the device**. Unplug the network and the feeder behaves identically — only the Agent layer stops. This is both a privacy claim and a reliability claim.
6. **Pocket-by-pocket mechanical dosing.** A continuously rotating paddle rotor replaces a flap: a flap's dose drifts with hopper level, whereas each rotor pocket has fixed volume, so `grams = pockets × grams-per-pocket` — making per-pet portioning true at the mechanical level too.

3.2 System Design

3.2.1 Overall architecture

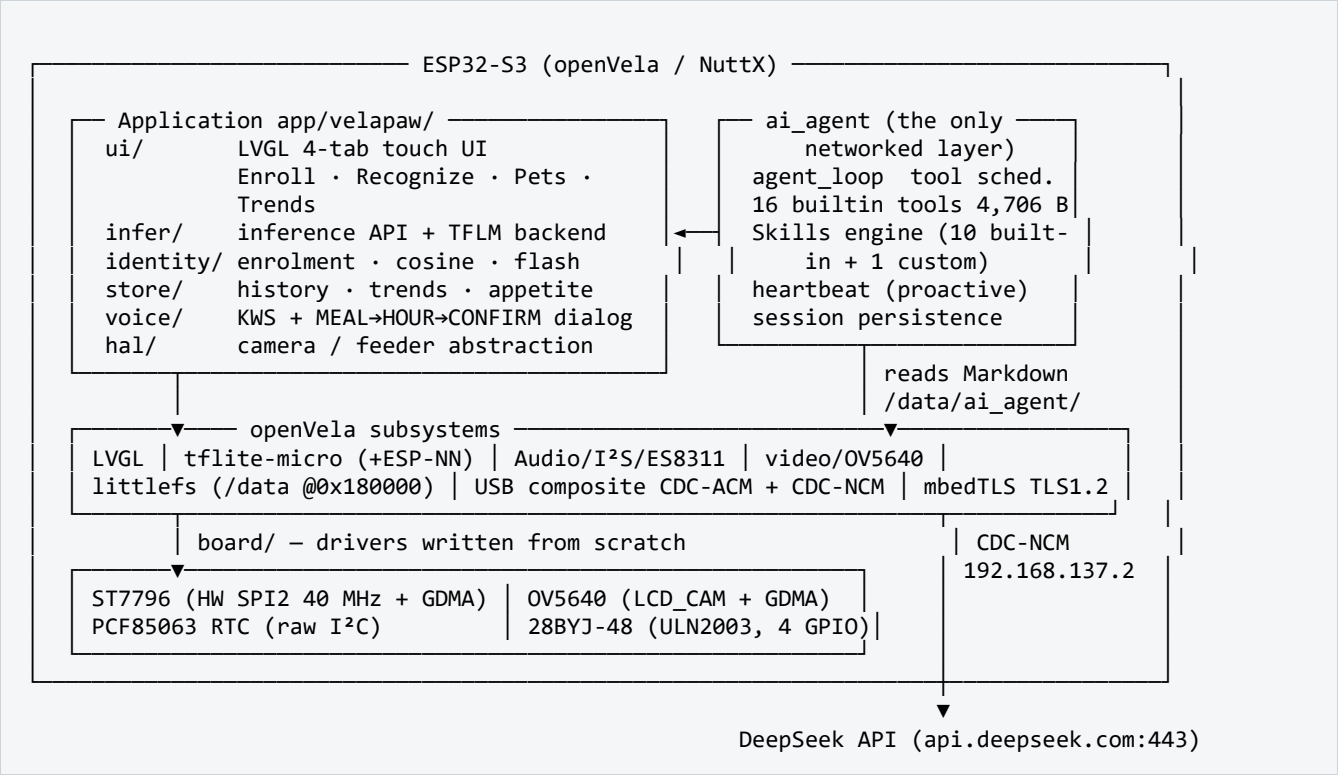

Figure 3-1 — VelaPaw system block diagram

3.2.2 Design decisions and rationale

Decision	Chosen	Rationale / rejected alternative
Recognition paradigm	Open-set metric learning (cosine match)	**Rejected N-way classification**: every new pet would require retraining and an OTA — unacceptable in a consumer product. Metric learning reduces "add a pet" to writing one flash record. The cost is inherently weaker rejection of non-pet objects, so the lever is placed on enrolment quality (close, well-lit, front-facing) rather than on the threshold.

Inference runtime	TFLite-Micro + ESP-NN	An early spike using the in-tree `tflm_benchmark` in a simulator confirmed **integration works, all operators are covered, and the tensor arena is only 84,420 B** (negligible against 8 MB PSRAM), and located the bottleneck in CONV_2D (84%). That determined the project would not be viable without LX7 SIMD kernels (ESP-NN).
Model storage	Raw flash partition, read into PSRAM at boot	**Rejected memory-mapped `const` arrays**: 1.44 MB far exceeds the DROM0 MMU window. Uses `spi_flash_read` instead; because a flash read suspends the cache, the reading thread's stack must live in internal SRAM (`.bss`), not PSRAM.
Dispenser mechanism	28BYJ-48 stepper + paddle rotor	**Rejected an MG90S servo + flap**: a servo can only oscillate, and with the inlet and outlet 180° apart a zero-net-rotation wobble can never carry a pocket across — it dispenses the initial charge and then spins uselessly. A flap's dose also drifts with hopper level, which would destroy the per-pet portioning claim outright. A stepper's continuous unidirectional rotation makes `grams = pockets × grams-per-pocket` strictly true, and its gearbox gives the low-speed torque needed to resist jamming.
Carrier board	Waveshare ESP32-S3-Touch-LCD-3.5-C	**Same SoC** as the official ESP32-S3-EYE, so the openVela port and all inference work carry over unchanged. But the product needs a **capacitive touchscreen** (enrolment, schedule editing, trends) and a **hardware RTC** (schedules stay correct across power loss) — the EYE has neither. Same platform, better carrier.
On-board networking	USB CDC-NCM composite device	**Rejected RNDIS**: Windows' `rndismp6.sys` hard-requires RNDIS on interface 0, which collides with the CDC-ACM console; the resulting Code-10 proved unavoidable. CDC-NCM makes the **console (COM port) and the network interface (`eth0`) usable simultaneously over one USB cable**. **Rejected on-board Wi-Fi as the demo link**: Wi-Fi works and remains in the configuration, but the USB composite link is far more reproducible in a demo setting.
Agent data bridge	Feeder writes Markdown; Agent reads it with `read_file`	**Rejected custom tools / IPC / shared memory.** After each feed the VelaPaw C application writes plain Markdown to `/data/ai_agent/velapaw/{status,feed_log}.md`, and the Agent reads it with the **framework's own `read_file`**. Zero new tools, zero new protocol, zero coupling: to the Agent it is an ordinary file; to the C application it is an `fprintf`.

Agent persistence	littlefs at `0x180000` mounted as `/data`	**Rejected tmpfs**: the API key, session history, and Skill files would have to be re-provisioned on every reboot. The littlefs partition starts at `0x180000` while `write-flash 0x0 nuttx.bin` writes only up to about `0x17ec80`, so **`/data` survives both reboots and firmware reflashes**.
Cloud model	DeepSeek `deepseek-chat`	OpenAI-compatible API with native function-calling support in the framework; directly reachable domestically, with controllable latency and cost.

3.2.3 Offline and degraded-network behaviour

This boundary is a deliberate design position, and it is what separates "AI hardware" from "connected hardware".

Function	**Offline**	**Notes**
Pet recognition (INT8 inference)	☑ Works	Model lives in flash/PSRAM; no network dependency whatsoever.
Recognition-gated feeding, daily cap	☑ Works	Purely local state machine plus RTC.
Body-condition scoring, history, trends	☑ Works	Second on-device model plus local littlefs history.
Voice meal scheduling (KWS)	☑ Works	On-device keyword model; never touches the network.
All touchscreen functions	☑ Works	Rendered locally by LVGL.
Agent `ask` Q&A	⛔ Stops	Requires the cloud LLM. On failure the console prints an explicit `[llm] API error` line; **no fabricated answer is produced**.
Agent proactive push	⛔ Silent	The heartbeat still fires and still reads the local files; only the LLM call fails. **Nothing is falsely reported and no data is lost** — once connectivity returns, the next heartbeat reasons over the complete history.

Degraded networks are handled at three layers. (1) *Transport* — `vela_tls` keeps and reuses a connection pool to `api.deepseek.com:443` (visible as `[vela_tls] Reusing pooled connection`). When the server closes an idle keep-alive the result is a zero-byte response, which the framework recovers from in-band; measured 3/3 recoveries. **This behaviour is not a defect.** (2) *Timeouts* — tuned to real MCU round-trips: `AGENT_LLM_TIMEOUT_SEC` raised to **420 s** and the socket timeout to **480 s**; the previous 60 s default discarded a 62.6 s answer that had in fact succeeded. (3) *Request size* — the tool cut described in §3.3.3, which brings the body within what the IOB pool can carry, and is by far the most effective of the three on a weak link.

3.2.4 Key modules

Module	Location	Responsibility and key design
UI	`app/velapaw/ui/` (`ui_lvgl.c`, 4,104 lines)	Four LVGL pages (Enroll / Recognize / My Pets / Trends); bilingual string table with live switching; the Trends page carries a coloured intake bar chart, an appetite line chart, and a body-condition gauge. Language is RAM-only (writing `0x818000` wedges the SoC and that address is a permanent no-go).
Inference	`infer/` (`backend_tflm.cc`, 399 lines)	Two independent `MicroInterpreters` (identity + body condition) with arenas in PSRAM. **Inference runs on a dedicated worker thread**, so the LVGL main loop hands off the frame and keeps rendering — touch and preview stay responsive through the 1.3 s inference. Recognition is **on-demand**, not continuous.
Identity	`identity/` (419 lines)	Enrolment store, cosine matcher (threshold 0.45), per-pet flash persistence (`0x800000`, magic `VPAW`), and pet avatars (`0x810000`, magic `VICN`).
Store / analytics	`store/` (`store_stub.c`, 625 lines)	30-day intake window, baseline and appetite percentage, daily cap, body-condition records; also generates the `status.md` / `feed_log.md` the Agent reads. **Key fix**: the day stamp originally advanced only when a demo button was pressed, so every feeding ever recorded piled into "today", intake was over-reported, and the feeder consequently *refused* to feed. Replaced with a VP02 fourth header word plus a clock-driven `vp_sync_day()`, with the rule that **the day stamp never moves backwards** (a build-date anchor makes every cold boot look like time travelling backwards).
Voice	`voice/` (`kws.cc` 822 + `mic.c` 2,370 lines)	A 14-label KWS model driving a constrained MEAL→HOUR→CONFIRM dialog. **Constrained decoding operates on the raw softmax and never renormalises.**

		Three gates at 0.40 / 0.55 / 0.55, plus a voting rule that rescues a dialog from three sub-threshold readings.
HAL	`hal/`	Abstractions for the camera (four backends: OV5640 / V4L2 / net / mock) and the feeder (stepper / mock), so the same application code runs on hardware and in simulation.
Agent layer	`agent/`	One custom Skill, one heartbeat task list, and three patches against `packages/ai_agent` (§3.4.5 and §3.3.3).

3.3 Core Algorithms and Principles

3.3.1 On-device models

Item	**Identity model**	**Body-condition model / KWS**
Task	Individual pet identification (open-set)	3-class body condition (thin / ideal / overweight); voice keyword spotting
Architecture	MobileNetV3-Small backbone + **TSFM** spatial-attention module + ArcFace metric head	Compact 3-class CNN; 14-label KWS convnet
Input / output	128×128 → L2-normalised **128-D embedding**	BCS: 3 logits; KWS: 14-class softmax
Framework	Training: TensorFlow / Keras (`host/`). Inference: **TensorFlow Lite for Microcontrollers** (`apps/mlearning/tflite-micro`)	
Quantisation	**INT8 post-training quantisation**; every operator is a TFLM built-in	
Model size	1.44 MB (flash `0x600000`)	626 KB (flash `0x760000`); the KWS model is embedded in firmware rodata
Operator set	ADD, CONV_2D, DEPTHWISE_CONV_2D, FULLY_CONNECTED, L2_NORMALIZATION, LOGISTIC, MEAN, MUL, PAD (9, all supported by the default resolver)	

Compute cost	Inference **≈1.3 s** with ESP-NN SIMD kernels versus **≈6.75 s** with TFLM reference kernels — a **≈5×** speedup, measured on real hardware. Tensor arena and operator-level timing come from the earlier TFLM spike (QEMU, comparable INT8 CNN, **not this model on S3**): arena **84,420 B** (non-persistent 55,296 B + persistent 29,124 B), split CONV_2D ≈84% / DEPTHWISE_CONV_2D ≈16%. The magnitudes are enough to establish that the arena is not the constraint and the convolution kernels are — which is what set the optimisation direction.
Memory placement	Both `.tflite` files are written to flash as raw blobs and read into PSRAM buffers by `spi_flash_read` at boot. The reading thread's stack must be in internal SRAM (`.bss`), because a flash read suspends the cache that PSRAM depends on.

ESP-NN integration and compliance. The 1.3 s figure depends on Espressif's ESP-NN (Apache-2.0) SIMD kernels, and that integration modifies the **public repository** `apps/mlearning/tflite-micro`. Per contest rules, changes to public repositories are submitted as a fork plus a PR against `dev-ai-contest-2026` and **are not merged into the team repository**. Consequently **this repository runs reference kernels out of the box (≈6.75 s)**, and ≈1.3 s once the ESP-NN fork is applied. The integration has three parts: (1) the library itself under `apps/mlearning/esp-nn/`; (2) seven kernel wrappers (conv, depthwise_conv, fully_connected, pooling, add, mul, softmax) placed in `.../micro/kernels/esp_nn/`; (3) a Makefile block compiling with `-DESP_NN=1 -DCONFIG_NN_OPTIMIZED -mlongcalls` that **filters out the matching reference kernels** to avoid duplicate symbols. Full wrapper source and reproduction steps are in `docs/esp-nn/`.

3.3.2 Cloud model (Agent layer)

Item	Detail
Model / provider	DeepSeek `deepseek-chat`
API	OpenAI-compatible Chat Completions with **function calling (tools)**; visible on the console as `[llm] OpenAI API with tools (model: deepseek-chat, NNNNN bytes)`
Endpoint / transport	`https://api.deepseek.com:443` via `vela_tls` + **mbedTLS, TLS 1.2**, with connection pooling
Authentication	Bearer API key. **The key lives in a config file on the device's littlefs `/data` partition — never in source, never in the repository.** (A plaintext key did leak into logs once during development; that was fixed, and `sk-*` / `ghp_*` / `Bearer *` redaction is enabled in the log collector.)

Link	On-board USB **CDC-NCM** network interface (`eth0`, `192.168.137.2`) → Windows host internet sharing → public internet. The same USB cable simultaneously carries the CDC-ACM console.
What is uploaded	**Text only**: system prompt, Skills summary, tool schemas, and numbers read from the device's local Markdown files. **No image or audio ever leaves the device.**
Timeouts	LLM 420 s / socket 480 s, tuned to real round-trips on this path

3.3.3 Key mechanisms

(1) The recognition-gated feeding state machine.

```
if  match score >= 0.45  and  the match is pet P
and there exists a meal M of P that is due, unfed today, and not skipped
and P's grams so far today + M's grams <= P's daily cap
then dispense (grams / grams-per-pocket) pockets
     -> record a feed event -> update status.md / feed_log.md
else show "next meal 18:00" on screen, dispense nothing
```

(2) Trimming the tool registry — the single highest-yield change in the project. The quantified form of the problem is in §3.1.2⑤. The fix excludes 20 tools irrelevant to a feeder via `s_excluded_tools` in `tool_registry.c` (4 wearable + 6 Feishu + 8 music + 2 quick-app), taking the tool count from 36 to **16** and the tool JSON to **4,706 B**. Reliability went from intermittent to **8/8**, and end-to-end latency from **150–300 s down to 3.6–35.6 s**.

> **The associated defect — an engineering lesson worth recording:** cutting a tool **leaves its callers behind**. This cut orphaned five natural-language fast-path rows, a hand-written `music_search` branch, and Feishu copy in every system prompt. Because `handle_nl_fast_path()` **runs before the LLM**, a fast path pointing at an unregistered tool returns a blank or wrong result **as the final answer**. The standing check is now: after any change to `s_excluded_tools`, grep for the removed tool names and require zero hits.

(3) Three classes of silent error. What they share is that *the answer reads fluently but did not come from the data*:

- **The heartbeat replayed its own history** — persisted session history survives reboots, so every firing after the first was reciting. `history_json` is `calloc`'d once *outside* the loop, so it needs an explicit `'\0'`.
- **`list_dir` prefix matching plus the local-tool shortcut** — the prefix was matched against an absolute path while the model passed a relative one, and because the tool sits in the local shortcut set, `(no files found)` was delivered to the owner as the answer. The fix **accepts both spellings** (the model actually uses three).
- **Stale console history suppressing tool calls** — the fix is not to trim history but to **insert a `system` message mid-array** forcing a re-read. The old history stays on disk untouched, and the result is still `iters=3 tools=2` with a correct answer.
- **`ask` truncating at the seventh word** — root cause was `MAX_ARGS 8` in `nsh_commands.c`, raised to **32**.

(4) Verification methodology. This project is deliberately distrustful of "it looks like it works", and settled on three criteria. `END status=ok iters=N tools=M` is the only real signal — a perfect answer with `iters=1 tools=0` is a history replay. `[skills] Skills summary: NNN bytes` is what proves a Skill loaded; `Skills system ready (N built-in)` is a compile-time constant and proves nothing. And to test whether a reply genuinely came from the remote model, **enrol a new pet mid-run** — if the next firing finds it, an in-RAM `llm_cache` replay is ruled out.

3.3.4 Depth of openVela usage

VelaPaw is not a thin application sitting on a BSP; it cuts vertically through openVela's graphics, AI, multimedia, filesystem, USB, and networking subsystems. **All three headline capability areas are used:**

Area	openVela components used	Depth
AI	`apps/mlearning/tflite-micro` (with `MATH_GEMMLOWP` / `MATH_RUY` / `MATH_KISSFFT` / `SYSTEM_FLATBUFFERS`)	The application links it directly and runs **two independent `MicroInterpreters`** (identity + body condition) with arenas in PSRAM and inference on a dedicated worker thread; ESP-NN LX7 SIMD kernels are wired into the same runtime (submitted as a fork/PR).
AI Agent	`packages/ai_agent` (`CONFIG_EXAMPLES_AI_AGENT_VELA`)	Runs on real hardware, not a simulator. Uses the tool system, the Skills engine, the heartbeat (proactive) channel, session persistence, and `vela_tls`; and contributes **3 patches, 18 files, +570 / −190 lines** back to the framework (§3.4.5).
Graphics	`CONFIG_GRAPHICS_LVGL` + `LV_USE_NUTTX` + `LV_USE_NUTTX_TOUCHSCREEN`, `LCD_FRAMEBUFFER`, `VIDEO_FB`	Four-page touch UI; a custom Noto Sans SC subset font (14/16/20) enabling live EN/中文 switching; hand-drawn bar, line, and gauge widgets on the Trends page (LVGL 9.1 has no `lv_meter`, so draw_dsc fields are used rather than getters).
Multimedia	`CONFIG_AUDIO` + `AUDIO_I2S` + `AUDIO_ES8311` + `ESP32S3_I2S0`, `SYSTEM_NXLOOPER`; `DRIVERS_VIDEO` + `VIDEO`	On-board microphone capture drives the KWS voice scheduler. **A real driver-layer defect was located and fixed: the ES8311 driver never programmed the RX TDM slot map — the root cause of the stalled RX channel.** Also established that `nxlooper` requires `CONFIG_AUDIO_DRIVER_SPECIFIC_BUFFERS`. The camera is brought into the video stack through the ESP32-S3 LCD_CAM peripheral with GDMA.

Filesystem	`ESP32S3_SPIFLASH_LITTLEFS`, `FS_PROCFS`, `FS_TMPFS`	The `/data` littlefs partition holds Agent config, Skills, sessions, and VelaPaw's feeding records, surviving both reboot and firmware reflash.
USB / networking	`USBDEV_COMPOSITE` + `CDCACM_COMPOSITE` + `CDCNCM_COMPOSITE`, `NET_TCP`, `NETDB_DNSCLIENT`, `CRYPTO_MBEDTLS`	A single USB cable enumerates both a console (CDC-ACM) and a network interface (CDC-NCM), which is what makes "real hardware + networked Agent + observable console" simultaneously true — the precondition for every piece of Agent evidence recorded for this submission.
Kernel / memory	`XTENSA_IMEM_USE_SEPARATE_HEAP`, `XTENSA_IMEM_REGION_SIZE=0x48000`, `ARCH_HAVE_EXTRA_HEAPS`, `IOB_NBUFFERS/NCHAINS=64`, `IOB_THROTTLE=24`	See the four platform-level fixes in §3.4.5.

3.3.5 Suggested improvements to openVela

All six come from real hardware experience on this project, ordered by value to the next developer:

1. **Provide an official optimised TFLM kernel path for Xtensa LX7 (ESP-NN).** The tree ships `cmsis-nn` (ARM) and `nnlib-hifi4` (HiFi DSP) but nothing for LX7 — despite the ESP32-S3 being this contest's primary chip. The gap is **6.75 s → 1.3 s (5×)**, which directly decides whether on-device vision AI is viable on openVela at all. It should be a Kconfig option in-tree rather than something every team forks for themselves.
2. **`mallinfo()` / `free` / `memdump` hang holding the heap lock on a freshly booted board and kill the whole chip.** `ai_agent` happens to call a heap walk as the very first statement of its agent task, so every `ask` froze the machine (initially misdiagnosed as a PSRAM problem). Suggestion: offer a non-blocking or timed variant of the heap-walk API, or at minimum document its locking semantics.
3. **`ARCH_HAVE_EXTRA_HEAPS` is a `selected` symbol and therefore never appears in a defconfig** — yet it is one of the three symbols required to keep task stacks out of PSRAM, and missing any one of them deadlocks the first `ask`. Suggestion: state the dependency explicitly in the `XTENSA_IMEM_USE_SEPARATE_HEAP` help text.
4. **The default IOB pool does not match `ai_agent`'s default request body.** 36 default tools produce a 19 KB request against roughly 7.8 KB of usable IOB — presenting as an intermittent hang with no timeout, which is the hardest failure class to diagnose. Suggestion: ship a leaner default tool set for MCU targets, or assert on request size at startup.
5. **`ai_agent`'s local-tool shortcut should be kept consistent with the tool registry.** `handle_nl_fast_path()` runs before the LLM, so once its target tool is excluded the wrong result is returned to the user as the final answer, with no warning. Suggestion: validate at compile time that the shortcut table is a subset of the registered tools.
6. **Several config options can silently destroy a board and deserve Kconfig warnings:** `CONFIG_RTC` (kills the USB console on this board), `CONFIG_NETINIT_THREAD` (hangs the whole board), `CONFIG_DEBUG_USB_INFO` (wedges the console). Likewise `CONFIG_NSH_LINELEN` **silently**

truncates an input line at about 80 characters, so provisioning a file with `echo … >> /long/path` writes corrupted content without any error.

3.4 Implementation

3.4.1 Software and firmware architecture

```
contest2026_043_CircuitForge/
├── app/velapaw/          device application (mapped into packages/demos/ by the manifest
linkfile)
│   ├── ui/               LVGL UI + i18n + CJK font        ui_lvgl.c 4,104 lines
│   ├── infer/            inference API + TFLM backend + two .tflite   backend_tflm.cc 399
│   ├── identity/         enrolment · cosine match · flash persistence  419
│   ├── store/            history · trends · appetite analysis          625
│   ├── voice/            KWS + dialog + mic capture   kws.cc 822 / mic.c 2,370
│   └── hal/              camera (4 backends) / feeder (2 backends)
├── agent/                ai_agent layer: custom Skill · heartbeat list · 3 framework patches
├── board/                board files: display/camera/RTC/stepper (esp32s3_st7789.c 961)
│                         auto-start (esp32s3_appinit.c 284) · bringup (658) · defconfig (205)
├── host/                 off-device training and export (TensorFlow/Keras, ~4,941 Python lines)
├── hardware/             3D-printed dispenser (OpenSCAD source + STL + BOM)
├── docs/                 engineering notes · benchmarks · ai_agent guide (all bilingual)
└── logs/                 AI-coding session logs (auto-collected, never hand-edited)
```

Hand-written C/C++ totals **13,049 lines** across `app/` and `board/` (excluding generated model arrays, font tables, and assets); host-side Python is about **4,941 lines**; patches against `packages/ai_agent` are **+570 / −190 lines**; the board defconfig is **205 lines**. The build artefact `nuttx.bin` is **1,561,920 B (1.49 MB)**.

Flash layout (16 MB)

Offset	Contents	Size	Notes
`0x000000`	Firmware `nuttx.bin`	1.49 MB	openVela + application + LVGL + TFLM + ai_agent
`0x180000`	`/data` (littlefs)	—	Agent config / key / Skills / sessions / feeding records; survives reflash
`0x600000`	Identity model `velapaw.tflite`	1.44 MB	Read into PSRAM at boot
`0x760000`	Body-condition model `bcs.tflite`	626 KB	Read into PSRAM at boot

`0x800000`	Enrolled pets (magic `VPAW`)	8 KB	Names · schedules · face embeddings
`0x810000`	Pet avatars (magic `VICN`)	32 KB	Card thumbnails

> **No-go address:** writing `0x818000` wedges the SoC and it is permanently banned in this project (which is why the UI language setting is RAM-only).

3.4.2 Data flow

```
[FEEDING PATH — entirely on-device, zero network dependency]

  OV5640 frame (LCD_CAM + GDMA)
        |
        v
  preprocess -> 128x128 INT8
        |
        v  [worker thread, ~1.3 s, UI never blocks]
  TFLM identity model --> 128-D L2-normalised embedding
        |
        v
  cosine match against enrolled pets --> (pet id, score, margin)
        |
        +-- score < 0.45 ----------------------> no dispense; UI shows "not recognised"
        |
        v score >= 0.45
  +------------- dual-condition gate -------------+
  | (1) does this pet have a meal due, unfed      |
  |     today, and not skipped?                   |
  | (2) grams today + meal grams <= daily cap?    |
  +--------+---------------------------+---------+
        both true                   either false
           |                            |
           v                            v
  28BYJ-48 rotates N pockets     UI shows "next meal 18:00"
  grams = pockets x grams/pocket
           |
           v
  append history (littlefs) -> update trends / appetite baseline / body condition
           |
           v
  write /data/ai_agent/velapaw/status.md    (per pet: grams today, meal count,
                                             7-day total, baseline, appetite %,
                                             body condition, daily limit)
        /data/ai_agent/velapaw/feed_log.md  (feed events with timestamp + match score)

[AGENT PATH — the only networked layer]

  (A) proactive: heartbeat service fires every N minutes
        |  injects /data/ai_agent/HEARTBEAT.md as a system message
        v
  (B) reactive: console  ask "how much did bob eat today"
        |
        v
  agent_loop --> system prompt + Skills summary (916 B) + tool schemas (4,706 B)
        |
        v  TLS 1.2 / mbedTLS / vela_tls -- CDC-NCM eth0 192.168.137.2 --> DeepSeek
        |
        v  model returns tool_calls
  list_dir -> read_file(status.md) -> read_file(feed_log.md)  [executed locally]
```

```
      |
      v  file contents fed back, request re-issued
final answer --> console / proactive push
      |          (if all is well: replies HEARTBEAT_OK only; the device stays quiet)
      v
session persisted to /data/ai_agent/sessions/tg_<chat_id>.jsonl
(console and heartbeat use separate namespaces)
```

Figure 3-2 — Data flow and processing

3.4.3 Hardware design and board bring-up

Was a new hardware platform adapted / were drivers developed? Yes. The Waveshare ESP32-S3-Touch-LCD-3.5-C has **no existing board support in openVela**. This project added a complete board implementation on top of `esp32s3-devkit` (`board/`, 1,903 lines of C plus a 205-line defconfig), **driving the following four peripherals from scratch**:

Peripheral	Connection	Driver highlights
ST7796 touch panel 320×480	Hardware SPI2 + GDMA at 40 MHz; CLK=GPIO5, MOSI=GPIO1, CS=GPIO4; FT5x06 touch on I²C0 (SCL=7 / SDA=8)	See "the hardest debug" below. Pixel rows must pass through an **internal-SRAM bounce buffer** — SPI DMA cannot read PSRAM cache-coherently. Address-window setup was batched from 11 SPI calls down to **5** (LVGL resets the window on every `putarea`).
OV5640 camera	ESP32-S3 **LCD_CAM** peripheral with a GDMA receive channel, configured over SCCB/I²C; brought out on a **30 cm FFC with a 1:1 coupler**	The preview was initially **transposed 90° and tinted violet** — two independent root causes. (a) Flip/mirror: bits 1 and 2 of `0x3820`/`0x3821` must be cleared, and the transposition itself can only be corrected in the software resample — the registers structurally cannot do it. (b) **The BLC pattern register `0x4514` must be set to `0x88`**. The discriminator is a reusable rule: **whole-frame hue is decisive — a flip or Bayer-phase error swaps red and blue and can never lose green, so violet (R+B, no G) means `0x4514`, not the flip bits**.
PCF85063 RTC	I²C0, **raw register access**	Deliberately does **not** use `CONFIG_RTC` — on this board it kills the USB CDC console. The boot time seed is designed as a **floor anchored to the build date** rather than a "anything before 2017 is invalid" plausibility test: the latter pins the board to one date forever with no way to correct it (`wall_set_min` only rewrites hours and minutes, and nsh has no `date`).
28BYJ-48 stepper feeder	ULN2003 driver board, IN1–IN4 = **GPIO9 / 10 / 11 / 43**; separate 5 V	Full-step sequence in software. **IN4 must be GPIO43 (U0TXD), not GPIO44 (U0RXD)** — GPIO44 is clamped and cannot drive the coil cleanly, which was the real cause of the early "buzzes but won't turn" (a pin problem, not a phase-order problem). GPIO43 is free

	supply, **common ground only**	because the console runs over USB-Serial/JTAG rather than UART0. **The stepper must never use GPIO12–16** — those are the ES8311 codec's private I²S bus (not brought out to any header) and are already claimed by the voice feature.
ES8311 audio	I²S0: MCLK=12, BCLK=13, DIN=14, WS=15, DOUT=16; 16 kHz	Getting microphone capture working required **locating and fixing a driver-layer defect: the RX TDM slot map was never programmed** — the true root cause of the stalled RX channel. A PSRAM cache trap additionally forced a two-slot stream over an 18-buffer pool, with the live slot flipping per capture.

The hardest debug: the hardware-SPI display. Full-screen writes worked, any partial window never appeared, and the identical byte sequence over bit-banging worked fine. It **survived roughly 60 rebuilds** — framebuffer mirroring, DMA on/off, software CS, PSRAM coherency, forced pin routing all had no effect.

- **Method — measure, don't guess.** The SPI pins are not brought out to any header, so the board firmware was made to **mirror `FSPID_OUT` and `FSPICLK_OUT` onto spare header pins** through the GPIO matrix, plus a trigger marker, and captured with a logic analyser (sigrok/PulseView + an FX2 clone). **The decisive first step was a ground-truth test**: drive four known square waves on the probe pins and refuse to trust any capture until they read cleanly. That step immediately exposed a wiring/grounding fault which had been the source of all the earlier "garbage" captures — only afterwards was the real capture trustworthy.
- **Finding.** The decode showed the ESP32 emitting **byte-perfect** `CASET 00 28 00 8B / RASET 00 28 00 8B / RAMWR / pixels`, with correct DC alignment, exactly eight clean clock edges per byte, and no glitches. **The peripheral was innocent all along.**
- **Root cause.** This panel's **chip select is hardwired low**, so it can never resynchronise its bit counter on a CS edge — it just keeps counting clocks. The firmware initialised the panel by bit-banging and only *then* handed the pads to the SPI peripheral; **that handover adds or drops a single clock edge**, and from that instant the panel is off by one bit. Every subsequent command decodes to garbage, `RAMWR` is never recognised, no pixels reach GRAM, and the screen keeps showing whatever bit-banging drew last — which looks *exactly* like "hardware SPI drew nothing".
- **Fix.** Hand the pads to SPI **first**, then **hardware-reset the panel** (a software reset will not do — the reset command itself would be misread), then run the entire init sequence over SPI. Once the panel resynchronises under SPI ownership, partial windows render correctly.
- **Payoff.** With the path open, the display left bit-banging behind for 40 MHz hardware SPI2 + GDMA: a full 480×320 repaint went from **≈246 ms to ≈61 ms (4×)**. A bug that looked like a dead end became a performance win.

Mechanical design (`hardware/`). Gravity hopper → paddle rotor → chute → bowl. Every part is parametric OpenSCAD source plus print-ready STL (PLA/PETG, 0.2 mm layers, 3 perimeters, no supports), with a JLC-outsourced parts list. The camera is removed from the mainboard and mounted, via a 30 cm FFC and a 1:1 coupler, inside a `cam_pod` on a two-legged `cam_mast` that straddles the chute — so **the screen faces the owner while the lens faces the pet**. **Safety warning: a reversed FFC is fatal at first power-on** — board pin N lands on camera pin 25−N, shorting a power rail straight onto data or ground pins. The camera module's ground is also **not internally**

common, so orientation cannot be confirmed by metering "through" the camera; it must be checked against the `1/24` silkscreen and bare-copper continuity (verified pin by pin on 2026-07-30).

3.4.4 Application and interaction design

There is no companion app, no account, and no cloud console — that is part of the product position, not a missing feature. All interaction happens on the device itself:

Surface	Design
Touchscreen (primary)	Four LVGL pages: **Enroll** (take a few close front-facing photos → store the mean embedding), **Recognize** (live preview + match result + dispense), **My Pets** (edit name, meals, grams, daily cap, per-meal skip; delete; avatar), **Trends** (intake bars, appetite line, body-condition gauge). The device **boots straight into the UI** (auto-started by `board_app_initialize()`) with no serial connection needed — it behaves like an appliance, not a dev board.
Voice (hands-free)	A three-step MEAL→HOUR→CONFIRM dialog, started by tapping on the "edit pet" page, using an on-device KWS model over a constrained vocabulary. Verified end-to-end on hardware (e.g. COMMIT meal 2 = 08:00).
Bilingual	Live 中文 / EN switching with a custom Noto Sans SC subset font. All UI strings, documentation, and README are bilingual.
Agent Q&A (console)	`ask <natural language>`, e.g. `ask how much did bob eat today`. Answers come from the device's own records, not canned text.
Agent proactive push	Heartbeat-triggered and **unattended**; speaks only when a threshold is crossed. This is the layer that means the owner does not have to walk over to the screen.

3.4.5 Custom Skill (mandatory for the AI Hardware Product Innovation track)

Skill file: `velapaw-feeding-digest.md` (VelaPaw Feeding Digest)
In repository: `agent/skills/velapaw-feeding-digest.md`
On device: `/data/ai_agent/skills/velapaw-feeding-digest.md`

> **Path note:** the track guide specifies `/data/agent/skills/` as the runtime Skill directory; this implementation uses **`/data/ai_agent/skills/`** because that is the directory the current `packages/ai_agent` framework *actually* scans on the device (evidenced by the console line `[skills] Skill exists: /data/ai_agent/skills/…`). The two are semantically equivalent; aligning with the specified directory is a mount-point rename.

Skill definition (full text):

```
# VelaPaw Feeding Digest
Report pet feeding, appetite and body condition.

## When to use
When asked about a pet, feeding, how much a pet ate, appetite,
body condition or feeder status. Also used by the periodic heartbeat check.

## How to use
1. read_file /data/ai_agent/velapaw/status.md
   Per pet it gives grams today, meal count, 7-day total, baseline,
   appetite percent, body condition, daily limit.
2. read_file /data/ai_agent/velapaw/feed_log.md
   Recent feed events with timestamps and match score.
3. Report per pet: grams today vs the daily limit, appetite vs baseline,
   and body condition.
4. Flag a concern ONLY if any of these is true:
   - appetite below 50 percent of baseline
   - grams today reached the owner daily limit
   - body condition is not ideal
   - no feed event today in the log
5. Otherwise say the pet is on track, one sentence.
6. Use only numbers from those files. Never invent.
```

Trigger scenarios (both verified on real hardware):

1. **Reactive — the owner asks.** A console query such as `ask how is jay doing today` matches *When to use*; the Agent runs `list_dir` → `read_file(status.md)` → `read_file(feed_log.md)` and answers from the actual numbers. Captured on hardware: `[llm] OpenAI API with tools (model: deepseek-chat, 11559 bytes)` → `END status=ok iters=3 tools=2 llm_ms=4340 elapsed=5s`, answering "*Today: 15 g in 1 meal / Appetite: 34% of baseline (44 g/day) / Owner daily limit: 60 g/day*".
2. **Proactive — the heartbeat health check** (the track's required "proactive + execution" scenario). The task item in `/data/ai_agent/HEARTBEAT.md` reads: *"Run the VelaPaw Feeding Digest skill. If it finds a concern, notify the owner with numbers. If all is on track, reply HEARTBEAT_OK only."* The heartbeat service (**the heartbeat service, not cron**) injects that file as a `system` message at a fixed interval and triggers a full agent loop. **Silence is the normal state** — a push happens only when one of the four thresholds in step 4 of the Skill is crossed. A captured **unattended** firing: `iters=4 tools=3` with four `read_file` calls and request bodies of **15,592 → 16,209 → 17,697 → 18,760 B**.

To prove this is genuinely running rather than replaying, a reproducible criterion is used: **enrol a new pet while the heartbeat is running**; if the next firing finds it, an in-RAM `llm_cache` replay is ruled out. That test passes — every firing is a real remote call. (There genuinely was a defect where firings after the first were served from cache; it is fixed by keying around `chat_id`.)

Four platform-level fixes (`agent/patches/`, 3 patches / 18 files / +570 −190 lines) — without them `ai_agent` does not run on this board at all:

#	Problem	Fix
1	The first `ask` **froze the entire chip**	Root cause: `agent_loop`'s pthread stack landed in **PSRAM (`0x3c…`)**, and littlefs' flash read disables the cache PSRAM depends on. Fixed with three Kconfig symbols: `XTENSA_IMEM_USE_SEPARATE_HEAP` + `XTENSA_IMEM_REGION_SIZE=`**`0x48000`** + the easily-missed

		`ARCH_HAVE_EXTRA_HEAPS`. **`0x30000` stops the freeze but is not shippable** — every task and pthread stack is carved from this heap, and ai_agent's task plus six pthreads on top of velapaw's 48 KB stack makes `pthread_create` return `ENOMEM`.
2	A heap walk kills a freshly booted board	`agent_loop_task`'s very first statement was `mallinfo()`, and `free`/`memdump`/`mallinfo` hang holding the heap lock on a fresh boot, killing the whole chip. Replaced with status reporting that does not walk the heap. (The "676 MB phantom node" was the broken walk itself, not memory corruption.)
3	IOB exhaustion hung requests silently	`IOB_NBUFFERS` / `IOB_NCHAINS = 64`, `IOB_THROTTLE = 24` (**160 chains is larger but will not boot**), plus a host→board ping to prime ARP first. The 1420 vs 1514 MTU difference is **real but not the cause** — `PKTSIZE` should not be "fixed".
4	An LLM timeout discarded a successful answer	`AGENT_LLM_TIMEOUT_SEC=420`, socket `480`. The old 60 s default threw away a 62.6 s answer that had actually succeeded, while presenting as "request timed out".

3.5 Testing and Results

3.5.1 Test environment

Item	Configuration
Hardware under test	Waveshare ESP32-S3-Touch-LCD-3.5-C (ESP32-S3-WROOM-1-N16R8: 8 MB octal PSRAM @80 MHz, 16 MB flash) + OV5640 on a 30 cm FFC + 28BYJ-48/ULN2003 on a separate 5 V ≥2 A supply (common ground) + the 3D-printed dispenser
Firmware	openVela / NuttX, `esp32s3-devkit:waveshare_lcd`; `nuttx.bin` 1,561,920 B
Build environment	Linux VM under VMware, `./build.sh esp32s3-devkit:waveshare_lcd -j4`; flashed from a Windows 11 host with `esptool`
Console / network	One USB cable: CDC-ACM console (`nsh>` / `vela>`) plus CDC-NCM interface `eth0` `192.168.137.2/24`, reaching the internet through Windows ICS
Cloud	DeepSeek `deepseek-chat`, TLS 1.2 / mbedTLS

Host-side evaluation	Model separability: validation set of **1,559 images across 39 individuals**, INT8 models, 50 latency samples

3.5.2 Functional tests

Case	Result	Criterion / evidence
Boots straight into the feeder UI	☑	No serial connection needed; auto-started by `board_app_initialize()`; visible throughout the demo video
Enrol a new pet (photos → mean embedding)	☑	On hardware; recognisable immediately after enrolment, **with no retraining and no reflash**
Recognise and feed per pet	☑	On hardware; dispenses only when recognition and schedule are both satisfied
Schedule gate: nothing before the due time	☑	UI shows "next meal 18:00"
At most once per meal per day; per-meal skip	☑	On hardware
Daily gram cap	☑	On hardware; confirmed by the Agent reading back "Owner daily limit: 60 g/day"
Body-condition scoring (3-class)	☑	On hardware, via the second TFLM interpreter
Intake trends + appetite-drop alerting	☑	Trends page bars/line/gauge; Agent reports "Appetite: 34% of baseline"
Stepper dispensing (pocket-quantised)	☑	On hardware (GPIO9/10/11/43); mechanism assembled
Voice scheduling MEAL→HOUR→CONFIRM	☑	End-to-end on hardware (e.g. COMMIT meal 2 = 08:00)

Live EN/中文 switching	☑	On hardware
Schedules / pets / avatars survive power loss	☑	Hardware RTC + flash persistence
Full feeder functionality offline	☑	Recognition, scheduling, dispensing, UI, and voice have no network dependency
`ai_agent` starts on hardware and loads the Skill	☑	`[tools] Tools JSON built (16 builtin + 0 providers)`, `[skills] Skill exists: /data/ai_agent/skills/…`, `[agent] Tools JSON loaded: 4706 bytes`
`ask` answers from real device data	☑	`END status=ok iters=3 tools=2` (`iters=1 tools=0` is treated as a replay and does not count as a pass)
Proactive push fires **unattended**	☑	Captured: `iters=4 tools=3` with four `read_file` calls
Proactive push stays silent when all is well	☑	Replies `HEARTBEAT_OK` only when no threshold is crossed
Skill refuses to invent numbers	☑	Skill rule 6, plus every number in the answer matching `status.md` item for item

3.5.3 Performance

Metric	**Measured**	**Baseline**	**Notes**
Identity inference latency	**≈1,310 ms**	≈6,750 ms (TFLM reference kernels)	ESP-NN LX7 SIMD kernels, **≈5× faster**. Runs on its own thread, so **the UI never blocks**.
Recognition rate (on-demand)	≈0.76 /s	≈0.15 /s	Recognition is triggered when a pet arrives at the bowl, not run continuously — a deliberate power and compute decision.

Full-screen repaint (480×320)	**≈61 ms**	≈246 ms (bit-banged GPIO)	40 MHz hardware SPI2 + GDMA, **≈4× faster**; window setup batched from 11 SPI calls to 5.
Tensor arena (early spike, QEMU)	**84,420 B**	204,800 B configured	Non-persistent 55,296 B + persistent 29,124 B. **Not an S3 measurement**, but sufficient to establish that the arena is not a constraint against 8 MB of PSRAM.
Compute split per inference (early spike, QEMU)	CONV_2D 84% DEPTHWISE 16%	—	643,819 / 122,680 / 768,105 ticks; `AllocateTensors` 24,860 ticks (one-off). **Not an S3 measurement**; used to decide that optimisation had to target the INT8 convolution kernels — a judgement later validated by the 5× speedup measured on S3.
Firmware size	**1,561,920 B**	16 MB flash	openVela + LVGL + TFLM + `ai_agent` + application; the two models occupy a further 2.05 MB of raw flash.
Agent end-to-end response	**3.6 – 35.6 s**	**150 – 300 s** (before the tool cut)	Typical captures: `llm_ms=4340 elapsed=5s`, `llm_ms=5260 elapsed=7s`.
Agent request body	11,312 → 12,667 → 14,008 B (three `ask` rounds)	19,391 B (single round, before)	Tool schemas cut from 9,946 B to **4,706 B**; Skills summary 916 B.
Agent buffers	ctx 8,192 / hist 8,192 / tool 8,192 B; tool output pool 2 × 16,384 B	—	Visible in the console startup log.
Network buffers	IOB 64 × 196 = 12,544 B (≈7,840 B	Default is insufficient	This ratio is the quantified root cause of the intermittent hang in §3.1.2⑤.

	usable after THROTTLE 24)		
Host-side model latency	fast 0.5 ms / accurate 0.6 ms (p95 0.6 / 0.7 ms)	—	**Relative host-CPU figures; not representative of the ESP32-S3.** Used only for A/B comparison.

3.5.4 Reliability and accuracy

(a) Recognition separability (host-side, 1,559 images / 39 individuals, INT8)

Metric	**fast model**	**accurate model (shipped)**
Model size	1,312 KB	**1,403 KB**
Operator count	8	9
Intra-class cosine	0.346	0.347
Inter-class cosine	0.103	**0.023**
Separation margin	0.243	**0.324**
EER threshold	0.130	0.070
Equal error rate (EER)	0.328	**0.291**

The accurate model was chosen on **separation margin** (0.324 vs 0.243) and the lower EER, at a cost of +91 KB and one extra operator — negligible against a 16 MB flash / 8 MB PSRAM budget.

> **An honest reading of this result:** EER 0.291 is a *lower-bound-style* figure measured on **39 individuals from a public pet dataset under open-set conditions**. It measures the difficulty of "given any two images, are they the same animal", **not the accuracy of the deployed product**. The real scenario is far easier: a household typically has 2–4 pets, enrolment photos are taken close-up and front-facing by the owner, and the match threshold is 0.45 — above the EER threshold, deliberately biased towards refusing rather than mis-feeding. The dominant factor in false accepts and false rejects **is enrolment quality, not the threshold**; that is a structural property of open-set metric learning, since a generic feature extractor inherently struggles to reject arbitrary non-pet objects, which is why the product puts its lever on guiding the user to take tight, well-lit, front-facing photos. **Work not yet done:** long-run false-accept / false-reject statistics for the three household pets under the final camera pose (`cam_mast` mount, current colour calibration). The current three pets' embeddings were captured on a firmware build that had the **violet colour cast**; after the `0x4514` fix

> they must be re-enrolled, and any earlier field accuracy figure is not comparable. This report therefore **gives no such number** rather than giving an untrustworthy one.

(b) Agent reliability

Metric	Before tool cut	After	Notes
Consecutive ask success rate	Intermittent failures (hang with no timeout)	**8 / 8**	Scored on the `END status=ok` line, not on whether the answer reads well
End-to-end latency	150 – 300 s	**3.6 – 35.6 s**	—
TLS interruption recovery	**3 / 3 recovered in-band**		Server closes an idle keep-alive → zero-byte response → `vela_tls` reconnects. **Confirmed not to be a defect.**
Proactive-push authenticity	**Every firing is a real remote call**		Cache replay ruled out by enrolling a pet mid-run and checking the next firing finds it

(c) Fault recovery and stability

- **Power loss:** the hardware RTC preserves time, and littlefs preserves pets, schedules, avatars, intake history, Agent sessions, and the API key. `/data` starts at `0x180000` while the firmware image ends around `0x17ec80`, so **even reflashing the firmware does not lose data**.
- **Network faults:** see the degradation table in §3.2.3. An LLM failure prints an explicit `[llm] API error NNN:` and **never produces a fabricated answer**.
- **Memory variation:** this project **deliberately reports no runtime heap-free figures** — for the reason in fix #2 of §3.4.5: on this board the heap-walk APIs hang holding the lock and kill the chip, and the fix was to report status without walking the heap. The heap numbers the firmware prints are therefore **assumed values, not measurements**, and quoting them as a metric would be misleading. The real memory constraint is the compile-time `XTENSA_IMEM_REGION_SIZE`: at `0x30000`, `pthread_create` returns `ENOMEM`; at `0x48000` the board reliably carries the ai_agent task, six pthreads, and velapaw's 48 KB stack. **That binary can/cannot-create-a-thread result carries more information than an untrustworthy free-bytes number.**
- **Continuous run time:** the longest controlled observation to date is the **600 s** unattended window during demo recording (`sleep 600`), through which the heartbeat fired on schedule, the console stayed clean, and the Agent answered normally. **Longer soak testing (24 h / 7 d) has not been performed**; it is listed in §3.7.3 and no extrapolation is made here.

- **Power consumption: not yet measured per-mode with a power analyser**, so no figures are given. The design basis for low power is that recognition is triggered on demand rather than run continuously (avoiding a 1.3 s inference occupying both cores back-to-back) and that the stepper is driven only briefly when the gate passes; the stepper's ≈240 mA per phase is carried by the separate 5 V supply and does not pass through the mainboard. Quantitative measurement is listed as future work.

3.5.5 Analysis

Mapping the results back onto the six challenges in §3.1.2:

1. ① **(compute) solved and quantified:** 6.75 s → 1.31 s. The key was profiling first to establish that 84% of the time was in CONV_2D, so the entire engineering effort could be aimed at the kernel library rather than scattered across parameters. **Measure first, optimise second** was this project's highest-return methodology.
2. ② **(no retraining) solved:** enrolment is a pure runtime operation, verified on hardware. The cost is weaker rejection than a closed-set classifier, which §3.5.4 discloses openly rather than glossing over.
3. ③ **(loading a large model) solved:** raw flash plus PSRAM, with the reusable lesson that a flash read suspends the cache — applicable to any ESP32-S3 + PSRAM + large-model project.
4. ④ **(SPI display) solved and converted into a 4× win:** the difference between 60 failed rebuilds and one successful measurement was not effort but **replacing guessing with observation**.
5. ⑤ **(request body vs IOB) solved, and the highest single-change yield:** reliability from intermittent to 8/8 and latency from 150–300 s to 3.6–35.6 s, purely by removing 20 unused tools. This lesson applies to every team running `ai_agent` on an MCU.
6. ⑥ **(silent errors) solved, and distilled into criteria:** one of the project's most valuable outputs is the set of three "read the log, not the prose" acceptance criteria (`END status=`, `Skills summary: NNN bytes`, and injecting new data mid-run). They are the tools that separate "looks like it works" from "is actually working".

3.6 AI-Native Development

This project was developed **with AI assistance throughout**. Session logs are collected automatically into `logs/KingMbewe/` by the organiser-provided `contest-log-collector` and are **never hand-edited**. All figures below were computed by script directly from that log directory and from the local raw session records.

Item	Data
Share of code written with AI	**≈100%** (measured as "produced or modified within an AI session"). The repository holds **13,049 lines** of hand-written C/C++ (`app/` + `board/`, excluding generated model arrays, CJK font tables, and assets), about **4,941 lines** of host-side Python, **+570 / −190 lines** of patches against `packages/ai_agent`, and a **205-line** board defconfig — roughly **18,800 lines** in total, all produced in AI pair-programming sessions and verified on real hardware iteration by iteration. The human share was: building and wiring the hardware, taking logic-analyser and multimeter measurements, performing every flash cycle and reading the resulting behaviour, and making directional decisions and falsifying AI conclusions — the

	several "this claim was later retracted" records in this report are the product of that human verification step.
AI tools used	**Claude Code** (CLI plus the VS Code extension) as the sole coding tool. Models appearing in the sessions: `claude-opus-5`, `claude-opus-4-8`, `claude-sonnet-5`, `claude-fable-5`. Log volume: **17 JSONL files / 78.7 MB / 55,255 collected events** — 7,559 user, 14,963 assistant, 32,733 tool — across **15 collection date directories from 2026-07-08 to 2026-09-04**. Tool-call distribution (leading entries): `Bash` 5,272, `PowerShell` 4,340, `Edit` 2,436, `Read` 2,263, `Grep` 713, `Write` 458, `AskUserQuestion` 247, `TodoWrite` 168, `ToolSearch` 126, `WebFetch` 89, `Glob` 79, `Artifact` 38, `ScheduleWakeup` 34, `WebSearch` 29, `Monitor` 22, `Skill` 17. That `Bash` and `PowerShell` together account for 9,612 calls reflects directly that this is a hardware project whose main loop is **build → flash → observe on real hardware**, not a pure software one.
MCP tool usage	**On-device: not used.** `CONFIG_AI_AGENT_MCP` is **explicitly disabled** in the board defconfig (along with Feishu / WeChat / MQTT / Node). This is a deliberate trade-off — as §3.3.3 shows, request size is this project's hardest constraint on an MCU, and any non-essential tool or protocol extension eats into the IOB budget. **Development side: two MCP connectors were configured** (a claude.ai connector and Google Drive) for reading design material and dataset references; because they require interactive OAuth authorisation, they were **never actually invoked** in this project's automated sessions. MCP usage is therefore honestly recorded as **0 calls**.
Skills used and added	**Runtime Skills added: 1** — `velapaw-feeding-digest` (`agent/skills/velapaw-feeding-digest.md` → `/data/ai_agent/skills/` on the device), satisfying this track's "at least one custom Skill" requirement; its definition and trigger scenarios are in §3.4.5. The device also carries the framework's **10 built-in Skills**, and the Skills summary in the system prompt is **916 B**. **Development-side Skills:** the organiser-provided set installed via the `.claude` manifest was used, of which the ones that materially contributed here were `contest-log-collector` (AI-coding log collection), `kconfig-tweak` (editing `.config` without menuconfig), `openvela-build` (build flow), `submit-pr` (submission flow), and the general-purpose `artifact-design`. The `Skill` tool fired **17 times** across the collected logs. **A persistent memory store of 40+ entries** was also maintained, recording across sessions which approaches had been *falsified on hardware* (for example "CONFIG_RTC kills the USB console", "writing `0x818000` wedges the SoC", "the claim that repeated wapi hangs the console is retracted"). This measurably reduced the cost of re-treading the same ground on a long-running project.
Total token usage	Summed from 35,074 assistant messages carrying usage records: Input tokens: **340,649** Output tokens: **44,262,781** Cache-creation tokens: **181,608,127** Cache-read tokens: **6,725,913,276**

	Total: 6,952,124,833 (≈6.95 billion) Excluding cache reads, the subtotal is **226,211,557 (≈226 million)**. Cache reads are 96.7% of the total, which is the characteristic shape of a long-context hardware-debugging project: every "change config → build → flash → read console" round carries the full board context back through inference.

What AI actually contributed, in order of value:

1. **Reframing an apparent software bug as a measurable physical one.** All 60 failed rebuilds on the hardware-SPI problem were looking for a cause in software. The turning point was the decision to mirror the SPI pins onto headers, run a ground-truth test first, and decode with a logic analyser. Designing that approach and working through it was a human-AI collaboration; the conclusion — *the peripheral is innocent, the problem is panel bit synchronisation* — was obtainable only by measurement.
2. **Discriminating between similar-looking symptoms.** For example: "whole-frame hue is decisive — a flip or Bayer-phase error can never lose green, so violet means `0x4514`, not the flip bits", which pinned in one step a problem that would otherwise have meant blind-testing dozens of registers.
3. **Establishing and applying falsification criteria.** Several early "conclusions" in this project were ultimately overturned and explicitly marked as retracted in the memory store (the "repeated wapi hangs the console" claim turned out to be a test-harness artefact; "stale history suppresses tool calls" came from one confounded A/B; "dead TX implies dead clock" was falsified at the register level). **Recording what was disproved matters as much as recording what was proved.**
4. **Debugging across the hardware-software boundary.** The missing ES8311 RX TDM slot map, the ratio between the IOB pool and the request body, the mutual exclusion between PSRAM stacks and the flash cache — none of these sit in a single layer; all required holding kernel configuration, driver registers, and application behaviour simultaneously.

3.7 Conclusions and Outlook

3.7.1 What was achieved

On a single ESP32-S3, on top of openVela, VelaPaw implements and **verifies on real hardware** every core capability of a multi-pet smart feeder:

- **On-device individual recognition with dual-condition gating** — open-set metric learning, so adding a pet needs no retraining; food is dispensed only when recognition and schedule agree.
- **On-device health monitoring** — 3-class body condition, a 30-day intake trend, and appetite-drop alerting.
- **Hands-free voice scheduling** — a three-step dialog driven by an on-device KWS model, verified end-to-end on hardware.
- **An on-device AI agent** — the `ai_agent` framework running on real hardware with one custom Skill, reactive `ask` plus proactive heartbeat push, **with an unattended firing captured**.
- **A new hardware platform brought up from scratch** — ST7796 display (40 MHz HW SPI + GDMA), OV5640 (LCD_CAM + GDMA), PCF85063 RTC, and the 28BYJ-48 stepper, plus a genuine driver-layer fix contributed for the ES8311 RX slot map.
- **A manufacturable mechanical design** — parametric OpenSCAD dispenser with STL and BOM, whose pocket quantisation makes per-pet portioning true mechanically.

- **Contributions back to openVela** — 3 patches / 18 files / +570 −190 lines, plus the ESP-NN integration submitted as a fork and PR.

Headline numbers: inference **6.75 s → 1.31 s (5×)**; UI repaint **246 ms → 61 ms (4×)**; Agent response **150–300 s → 3.6–35.6 s** at **8/8** reliability.

3.7.2 Market outlook and commercial value

Market signal. In March 2026 Xiaomi launched the Mijia Smart Pet Feeder 2 Vision Edition (¥449 crowdfunded / ¥549 retail) — a mass-market product built specifically to bring a camera into the feeding scenario. That is the company with the best read on what pet owners will pay for, independently validating VelaPaw's founding judgement with real money: **the camera belongs on the feeder**. VelaPaw takes the same judgement one step further — from "see the bowl" to **"know which one it is"**, entirely on-device.

Target users, in order of willingness to pay:

1. **Multi-pet households (core).** The sharpest pain: portion management simply does not work with more than one animal. This is the one group whose problem *cannot* be solved without individual recognition.
2. **Owners managing prescription diets or weight loss.** A vet's gram-level plan has to actually be executed and recorded; VelaPaw's daily caps plus per-pet history are exactly that record of execution.
3. **Privacy-sensitive owners.** People unwilling to have household camera footage enter any cloud account. "Images never leave the device" is a hard property here, not a privacy-policy clause.
4. **Carers of senior pets.** A drop in appetite is the earliest signal of many illnesses; proactive push turns "the owner must log this daily" into "the device speaks up when something is wrong".

Business model. The core is **one-time hardware sales with no subscription** — a structural advantage that comes directly from on-device AI: with no cloud inference cost, there is no cost to amortise through a subscription.

Revenue stream	Notes
Hardware sales (primary)	Positioned against the ¥449–549 band of camera-equipped feeders. The BOM is essentially an ESP32-S3 module, a 3.5″ touch panel, an OV5640, a stepper, and injection-moulded parts — at volume the cost structure is comparable to existing vision-equipped feeders, while **eliminating the ongoing cost of cloud inference and storage**.
Consumables and accessories	Washable hoppers, rotors of differing capacity (changing one `WIDTH` parameter changes grams-per-pocket), spare bowls.

Optional cloud add-on (not required)	The Agent layer's health digest could become an optional subscription (long-term trend reports, vet-readable exports). **Cancelling it leaves every core function intact** — a clear contrast with competitors that brick without a subscription.
B2B extension	Multi-cage portioned feeding and intake records for veterinary clinics and boarding facilities — fundamentally the same "identify, portion, record" capability.

Differentiators: (1) individual recognition rather than merely "seeing"; (2) no app, no account, no cloud — it works out of the box; (3) full functionality offline; (4) mechanically guaranteed pocket-level dosing; (5) proactive health monitoring rather than passive viewing.

3.7.3 Limitations and future work

Listed **honesty-first**, including the items for which this report deliberately gives *no* number:

Limitation	Status and plan
Field recognition accuracy not quantified	The three enrolled pets' embeddings were captured on a firmware build with the **violet colour cast**; after the `0x4514` fix they must **all be re-enrolled**, and earlier field data is not comparable. Plan: after re-enrolment, log every match over two weeks under the final camera pose and report false-accept / false-reject rates plus a threshold ROC.
No long-duration soak test	Longest controlled observation is a 600 s unattended window. Plan: 24 h and 7 d continuous runs, recording heartbeat firing success rate, TLS reconnection count, littlefs write count and wear, and whether any slow resource leak appears.
Power consumption not measured	No per-mode power-analyser data yet. Plan: measure average and peak current for standby (screen on / off), a single inference, and a dispense cycle.
Runtime heap usage not observable	Because the heap-walk APIs hang this board, status is currently reported with assumed values. Plan: implement a lightweight lock-free heap statistic (or interval sampling) so memory variation becomes a quantifiable metric — which is also improvement suggestion #2 to openVela.
ESP-NN not vendored in the team repo	Submitted as a fork/PR per the rules, so this repository runs reference kernels (6.75 s) out of the box. Plan: publish the PR link in the README once merged, and provide a one-command apply script.

Weak rejection of non-pet objects	A structural property of open-set metric learning. Plan: add a very lightweight "is this a pet face" binary gate ahead of embedding matching, which should measurably reduce mis-dispensing.
No always-on voice wake	Voice scheduling currently requires a tap to start. Plan: if always-on wake is added, it will use the contest's mandated wake word (*你好，openvela* / *Hello，openvela*) exactly.
Skill runtime directory differs from the spec	The implementation uses `/data/ai_agent/skills/` (the directory the framework actually scans); the spec says `/data/agent/skills/`. Plan: align the mount point once the framework settles on one.
Agent is console-only	`ask` currently runs over the NSH console. Plan: surface the Agent's answers and pushes on a "Health" page on the touchscreen, so the owner never needs a serial connection.
